# R.M.K
# GROUP OF ENGINEERING INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

# R.M.K
# GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

## Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

# 22IT957

# DEVOPS

| | | |
|---|---|---|
| **Department** | **:** | **Information Technology** |
| **Batch/Year** | **:** | **2023-2027/ III** |
| **Created by** | **:** | **Dr. A. ANNA LAKSHMI, ASP/IT, RMKEC** |
| | | **Ms.N.ABIRAMI** |
| | | **AP/IT, RMDEC** |
| **Date** | **:** | **06-01-2026** |

# 1.TABLE OF CONTENTS


| S.NO. | CONTENTS | SLIDE NO. |
|---|---|---|
| 1 | CONTENTS | 5 |
| 2 | COURSE OBJECTIVES | 7 |
| 3 | PRE REQUISITES (COURSE NAMES WITH CODE) | 8 |
| 4 | SYLLABUS (WITH SUBJECT CODE, NAME, LTPC DETAILS) | 9 |
| 5 | COURSE OUTCOMES (6) | 12 |
| 6 | CO- PO/PSO MAPPING | 13 |
| 7 | LECTURE PLAN –UNIT 4 | 15 |
| 8 | ACTIVITY BASED LEARNING –UNIT 4 | 16 |
| 9 | CROSSWORD PUZZLE | 17 |
| 10 | VIDEO LINK-QUIZ | 18 |
| 11 | TEST YOURSELF | 19 |
| 12 | **LECTURE NOTES – UNIT 4** | 20 |
| 13 | Docker Introduction | 20 |
| 14 | Images, Containers | 23 |
| 15 | Docker Hub | 32 |
| 16 | Links, Volume, Network, Interactive Sessions | 33 |
| 17 | K8 - Single and Cluster Mode | 63 |
| 18 | Secrets, Persistence Volume and Claim | 55 |
| 19 | Replica Factor, Services, | 52 |
| 20 | Pods, Deployments, logs, Kubernetes | 51 |

| S.NO. | CONTENTS | SLIDE NO. |
|---|---|---|
| 21 | ASSIGNMENT 1- UNIT 4 | |
| 22 | PART A Q & A (WITH K LEVEL AND CO) | 65 |
| 23 | PART B Q s (WITH K LEVEL AND CO) | 66 |
| 24 | SUPPORTIVE ONLINE CERTIFICATION COURSES | 67 |
| 25 | REAL TIME APPLICATIONS IN DAY TO DAY LIFE AND TO INDUSTRY | 68 |
| 26 | CONTENTS BEYOND THE SYLLABUS | 69 |
| 27 | ASSESSMENT SCHEDULE | 71 |
| 28 | PRESCRIBED TEXT BOOKS & REFERENCE BOOKS | 70 |
| 29 | MINI PROJECT SUGGESTIONS | 72 |


R.M.K GROUP OF INSTITUTIONS

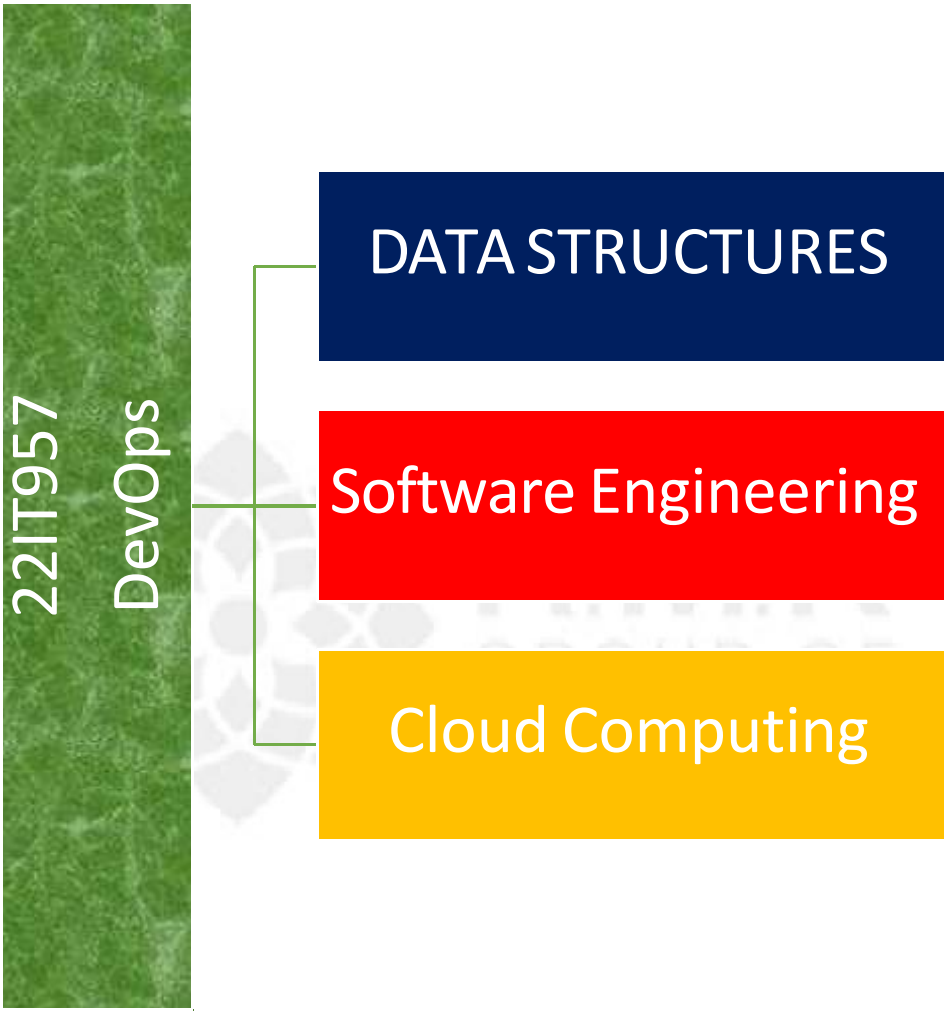
# 2. COURSE OBJECTIVES

- **OBJECTIVES:**
  - Understand the concepts of DevOps and the issues it resolves
  - Learn the DevOps tools set
  - Learn to Develop automation using Maven
  - Understand Continuous Delivery and Continuous Deployment
  - Understand Docker Containerization

R.M.K GROUP OF INSTITUTIONS

# 3. PRE REQUISITES

## PRE-REQUISITE CHART



**4. 22IT957** **DEVOPS (Lab Integrated)** **L T P C**
**2 0 2 3**

**OBJECTIVES:**

The Course will enable the learners:

- Understand the concepts of DevOps and the issues it resolves
- Learn the DevOps tools set
- Learn to Develop automation using Maven
- Understand Continuous Delivery and Continuous Deployment
- Understand Docker Containerization

**UNIT I INTRODUCTION TO DEVOPS 6+6**

What Is DevOps? Culture, principles & benefits, Architecture, SDLC Life Cycle & DevOps LifeCycle, Workflow and Principles, Tools, CI, CD and CD Pipelines – Version Control Systems (e.g., Git, GitHub/GitLab/Bitbucket) Linux Introduction Basic Commands, Scripting – Toolsets – SSH, Curl, CAT, Sudo, Man, apt-get *

,

**List of Exercises:**

1. Prepare an application preferably in Spring Boot with following features like

a. Maven Build
b. Unit Testing
c. Application Functionality
d. DockerFile
e. App Configuration
f. And finally, the application checked in to GitHub Repo

2. Install Jenkins with following plugins

a. Maven
b. Java
c. GitHub
d. Artifact Repo
e. Pipeline
f. SonarQube

**4. 22IT957** **DEVOPS (Lab Integrated)** **L T P C**
**2 0 2 3**

**UNIT II** **DevOps TOOLCHAIN BASICS** **6+6**

Maven Build Management, Goals, Profiles, Plugins, LifeCycles, Configuration, Parent/Child - SCM Tools - GitHub, Init, CheckIn, Merge, Pull, Push, Local and Remote Repo, Pull Request, Tagging Strategy – Unit Testing – Unit Testing scripts - Artifact Repository - Release Management aligned Repos, Private and Public Repos Monitoring - Tools like nagios to assist in monitoring and managing the deployed instances.

**List of Exercises:**

1. Configure and Setup a CI in Jenkins with the application developed earlier. And enable
following configurations.
a. Trigger the CI – GitHub Hooks
b. Generate the CI Report and publish the same as Email
c. Test Case Report ,
d. Dynamic Build ID Generation
e. Post and Pre build activities.
2. Create Automated Unit and Functional (QA) Testing Scripts using TestNG & Selenium

**UNIT III** **TESTING AUTOMATION** **6+6**

Maven with Unit / Integration / Performance Testing - Report Generation and Configuration.

**List of Exercises:**

1. Configure and Setup a CD in Jenkins with the application developed earier.
a. Enable JenkinsFile for automating the build process
b. Enable Unit testing automation in the CD Pipeline.
c. Enable Functional testing automation in the CD Pipeline.

**UNIT IV** **DEPLOYMENT AND MONITORING – DOCKER** **6+6**

Jenkins Setup – Configuration – CD – Pipelines – Schedule and Hooks Run. Docker Introduction, Images, Containers, Docker Hub, Links, Volume, Network, Interactive Sessions , Docker REST API K8 – Helm Framework, Single and Cluster Mode, Secrets, Persistence Volume and Claim, Replica Factor, Services, Pods, Deployments, logs, Kubernetes.

**4. 22IT957** **DEVOPS (Lab Integrated)** **L T P C**
**2 0 2 3**

**List of Exercises:**

1. Install and Configure Docker
2. Configure and Setup a CD in Jenkins with the application developed earier.

- Add DockerFile Build Process for the application in scope.
- Build and Deploy the same
- Add the Docker Clean up, Image Creation, and creating instance of Containers via CD Process. (using pre-build scripts)

**UNIT V DEPLOYMENT AND MONITORING - JENKINS 6+6**

Jenkins - Setup and Configuration, Jobs - Continuous Integration, Continuous Delivery and Continuous Deployment Configuration
Jenkins CD->Integration with SonarQube, Artifact Repository, Docker Hub & Kubernetes via REST API , Logging and Montorting - Prometheus, Grafana, Nagios. ,

**List of Exercises:**

1. Install and Nagios
2. Apply monitoring principles to measure

a. Build Time
b. Unit & Integration Testing Time
c. Resources Consumed by Jenkins
d. Deployment Time
e. Analyze the logs files for any anomalies
f. Verifying ACLs and Security based access rights.
g. Network Performance
h. Overall Serer Storage, Performance etc

**TOTAL: 30+30=60 PERIODS**

# 5.COURSE OUTCOME

| Course Code | Course Outcome Statement | Cognitive/ Affective Level of the Course Outcome | Expected Level of Attainment |
|---|---|---|---|
| **Course Outcome Statements in Cognitive Domain** | | | |
| C702.1 | Understand the concept of DevOps Architecture. | Apply K3 | 60% |
| C702.2 | Apply the DevOps Tools in real time applications | Analyse K4 | 60% |
| C702.3 | Build Maven with Unit, Integration and Performance Testing | Understand K2 | 60% |
| C702.4 | Deploy and monitor using Docker | Apply K3 | 60% |
| C702.5 | Deploy and monitor using Jenkins | Analyse K4 | 60% |
| C702.6 | Develop project with CI CD Pipeline, identify project costs and efforts required | Apply K3 | 60% |
| **Course Outcome Statements in Affective domain** | | | |
| C702.7 | Attend the classes regularly | Respond (A2) | 95% |
| C702.8 | Submit the Assignments regularly. | Respond (A2) | 95% |
| C702.9 | Participation in Seminar/Quiz/ Group Discussion/ Collaborative learning and content beyond syllabus | Valuing (A3) | 95% |

# 6.CO-PO/PSO MAPPING

## Correlation Matrix of the Course Outcomes to Programme Outcomes and Programme Specific Outcomes Including Course Enrichment Activities

| Course Outcomes (Cos) | | Programme Outcomes (POs), Programme Specific Outcomes (PSOs) | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 | PSO1 | PSO2 | PSO3 |
| | | K3 | K4 | K5 | K5 | K3/K5 | A2 | A3 | A3 | A3 | A3 | A3 | A2 | K3 | K3 | K3 |
| C305.1 | K3 | 3 | 2 | 1 | 1 | 3 | | | | | | 3 | | 3 | 3 | 3 |
| C305.2 | K4 | 3 | 3 | 2 | 2 | 3 | | | | | | 3 | | 3 | 3 | 3 |
| C305.3 | K2 | 2 | 1 | | | | | | | | | 2 | | 3 | 3 | 3 |
| C305.4 | K3 | 3 | 2 | 1 | 1 | 3 | | | | | | 3 | | 3 | 3 | 3 |
| C305.5 | K4 | 3 | 3 | 2 | 2 | 3 | | | | | | 3 | | 3 | 3 | 3 |
| C305.6 | K3 | 3 | 2 | 1 | 1 | 3 | | | | | | 3 | | 2 | 2 | 2 |
| C305.7 | A2 | | | | | | | | | | | | 3 | | | |
| C305.8 | A2 | | | | | | | | 2 | 2 | 2 | | 3 | | | |
| C305.9 | A3 | | | | | | 3 | 3 | | 3 | 3 | | 3 | | | |
| C305 | | 3 | 3 | 2 | 2 | 3 | 1 | 1 | 1 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |

# UNIT 4

# DEPLOYMENT AND MONITORING - DOCKER

R.M.K GROUP OF INSTITUTIONS

# LECTURE PLAN – UNIT IV

| | .NIT I SOFTWARE PROCESS AND AGILE DEVELOPMENT | | | | | | |
|---|---|---|---|---|---|---|---|
| Sl. No | TOPIC | NO OF PERIODS | PROPOSED LECTURE PEROID | ACTUAL LECTURE PERIOD | PERTAINING CO(s) | TAXONOMY LEVEL | MODE OF DELIVERY |
| 1 | Docker Introduction | 1 | 17.02.2025 | | CO4 | K2 | MD2 |
| 2 | Images, Containers | 1 | 19.02.2025 | | CO4 | K2 | MD2 |
| 3 | Docker Hub, Links. | 1 | 20.02.2025 | | CO4 | K2 | MD2 |
| 4 | Volume, Network, Interactive Sessions - K8 | 1 | 25.02.2025 | | CO4 | K3 | MD2 |
| 5 | Single and Cluster Mode, Secrets, | 1 | 26.02.2025 | | CO4 | K3 | MD1 |
| 6 | Persistence Volume and Claim | 1 | 27.02.2025 | | CO4 | K4 | MD2 |
| 7 | Replica Factor, Services, | 1 | 28.02.2025 | | CO4 | K3 | MD2 |
| 8 | Pods, Deployments | 1 | 04.03.2025 | | CO4 | K3 | MD2 |
| 9 | logs, Kubernetes | 1 | 05.03.2025 | | CO4 | K4 | MD2 |

# LECTURE PLAN – UNIT 4

## ASSESSMENT COMPONENTS

- AC 1. Unit Test
- AC 2. Assignment
- AC 3. Course Seminar
- AC 4. Course Quiz
- AC 5. Case Study
- AC 6. Record Work
- AC 7. Lab / Mini Project
- AC 8. Lab Model Exam
- AC 9. Project Review

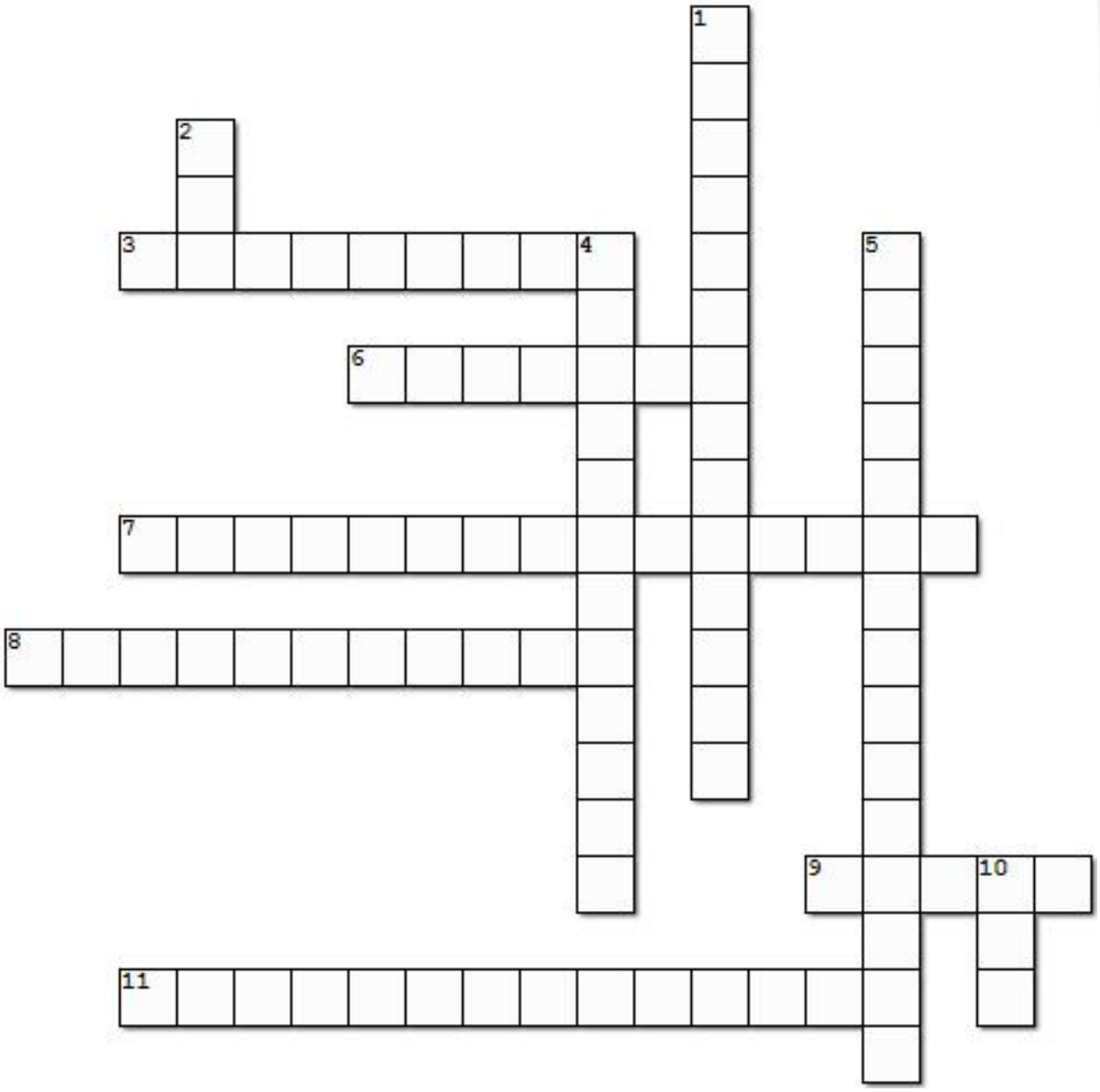
## MODE OF DELEIVERY

MD 1. Oral presentation
MD 2. Tutorial
MD 3. Seminar
MD 4 Hands On
MD 5. Videos
MD 6. Field Visit

R.M.K GROUP OF INSTITUTIONS

# ACTIVITY BASED LEARNING – UNIT 4

## COMPLETE THE PUZZLES GIVEN BELOW




**Down**

1. lightweight, efficient, low-risk, flexible, predictable approach
2. In XP, User requirements are expressed as
3. Inception,elaboration,construction,deployment
5. critical safety systems & defect free systems
6. Project risk factor is considered in which model

**Across**

4. XP design
7. The spiral model was originally proposed by
8. Requirement well understood
9. Stand-up meetings
10. Modelling tool
11. Process of correcting errors

# ACTIVITY BASED LEARNING – UNIT 4

## QUIZ- LINK

### Unit 4:

https://quizizz.com/admin/quiz/6417c38f432a57002135873c/devops?fromSearch=true&source=null

DEVOPS VIDEO link

https://youtu.be/slajN4ox7Eo?si=sCcz6AGsjvOBiimp

## Test Yourself

1. **When was DevOps introduced?**

- A) 1999 and 2000
- B) 2003 and 2004
- C) 2007 and 2008
- D) 2010 and 2011

2 . **Who is the father of DevOps?**

- A) Patrick Debois
- B) Gene Kim
- C) David Farley
- D) Bridget Kravchenko

3 **What is the full form of Devops?**

- A) Drive and Operations
- B) Digital and Operations
- C) Development And Operations
- D) None of the above

4. **Identify the tool for DevOps?**

- A) Monit
- B) Nagios
- C) Jenkins
- D) All of the above

R.M.K GROUP OF INSTITUTIONS

# UNIT 4
# DEPLOYMENT AND MONITORING - DOCKER

## Docker Introduction

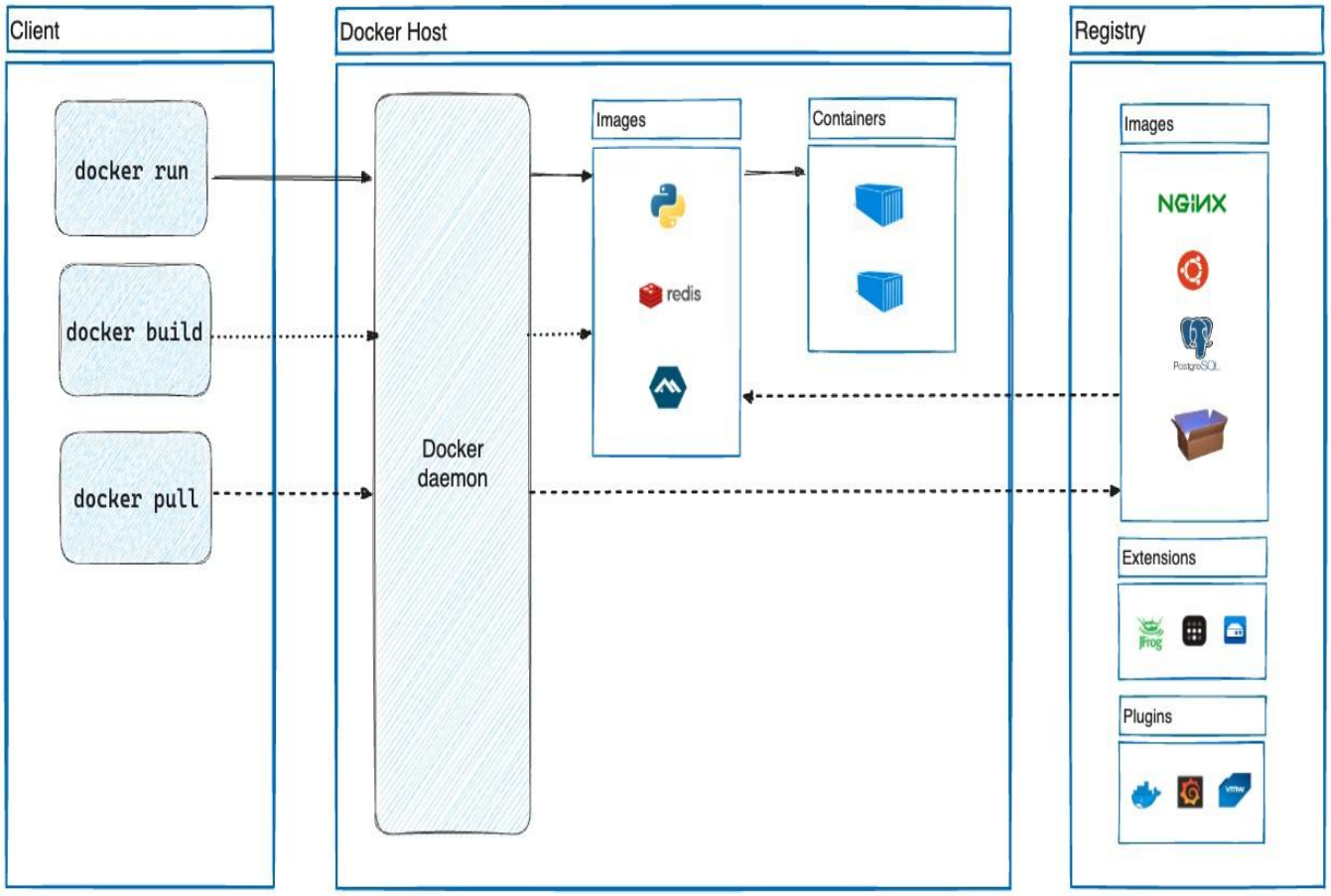
- Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code, you can significantly reduce the delay between writing code and running it in production.
- The Docker platform

  Docker provides the ability to package and run an application in a loosely isolated environment called a container. The isolation and security lets you to run many containers simultaneously on a given host. Containers are lightweight and contain everything needed to run the application, so you don't need to rely on what's installed on the host. You can share containers while you work, and be sure that everyone you share with gets the same container that works in the same way.
- Docker provides tooling and a platform to manage the lifecycle of your containers:
- Develop your application and its supporting components using containers.
- The container becomes the unit for distributing and testing your application.

  When you're ready, deploy your application into your production environment, as a container or an orchestrated service. This works the same whether your production environment is a local data center, a cloud provider, or a hybrid of the two.

# Docker architecture

Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface. Another Docker client is Docker Compose that lets you work with applications consisting of a set of containers.



- The Docker daemon
- The Docker daemon (dockerd) listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes. A daemon can also communicate with other daemons to manage Docker services.
- The Docker client
- The Docker client (docker) is the primary way that many Docker users interact with Docker. When you use commands such as docker run, the client sends these commands to dockerd, which carries them out. The docker command uses the Docker API. The Docker client can communicate with more than one daemon.
- Docker Desktop
- Docker Desktop is an easy-to-install application for your Mac, Windows or Linux environment that enables you to build and share containerized applications and microservices. Docker Desktop includes the Docker daemon (dockerd), the Docker client (docker), Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper.
- Docker registries
- A Docker registry stores Docker images. Docker Hub is a public registry that anyone can use, and Docker looks for images on Docker Hub by default. You can even run your own private registry.
- When you use the docker pull or docker run commands, Docker pulls the required images from your configured registry. When you use the docker push command, Docker pushes your image to your configured registry.
- Docker objects
- When you use Docker, you are creating and using images, containers, networks, volumes, plugins, and other objects. This section is a brief overview of some of those objects.

## Images

- An image is a read-only template with instructions for creating a Docker container. Often, an image is based on another image, with some additional customization. For example, you may build an image which is based on the ubuntu image, but installs the Apache web server and your application, as well as the configuration details needed to make your application run.
- You might create your own images or you might only use those created by others and published in a registry. To build your own image, you create a Dockerfile with a simple syntax for defining the steps needed to create the image and run it. Each instruction in a Dockerfile creates a layer in the image. When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt. This is part of what makes images so lightweight, small, and fast, when compared to other virtualization technologies.
- Containers
- A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI. You can connect a container to one or more networks, attach storage to it, or even create a new image based on its current state.
- By default, a container is relatively well isolated from other containers and its host machine. You can control how isolated a container's network, storage, or other underlying subsystems are from other containers or from the host machine.
- A container is defined by its image as well as any configuration options you provide to it when you create or start it. When a container is removed, any changes to its state that aren't stored in persistent storage disappear.

Example docker run command

The following command runs an ubuntu container, attaches interactively to your local command-line session, and
runs /bin/bash

- $ docker run -i -t ubuntu /bin/bash
- When you run this command, the following happens (assuming you are using the default registry configuration):
- If you don't have the ubuntu image locally, Docker pulls it from your configured registry, as though you had run docker pull ubuntu manually.
- Docker creates a new container, as though you had run a docker container create command manually.
- Docker allocates a read-write filesystem to the container, as its final layer. This allows a running container to create or modify files and directories in its local filesystem.
- Docker creates a network interface to connect the container to the default network, since you didn't specify any networking options.
  This includes assigning an IP address to the container. By default,
- containers can connect to external networks using the host machine's network connection.

  Docker starts the container and executes /bin/bash. Because the container is running interactively and attached to your terminal (due to the -i and -t flags), you can provide input using your keyboard while Docker logs the output to your terminal.
- When you run exit to terminate the /bin/bash command, the container stops but isn't removed. You can start it again or
- remove it.

## What is a container?

A container is a sandboxed process running on a host machine that is isolated from all other processes running on that host machine. That isolation leverages kernel namespaces and cgroupsopen_in_new, features that have been in Linux for a long time. Docker makes these capabilities approachable and easy to use. To summarize, a container:

- Is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI.
- Can be run on local machines, virtual machines, or deployed to the cloud.
- Is portable (and can be run on any OS).
- Is isolated from other containers and runs its own software, binaries, configurations, etc.
- If you're familiar with chroot, then think of a container as an extended version of chroot. The filesystem comes from the image. However, a container adds additional isolation not available when using chroot.
- What is an image?
- A running container uses an isolated filesystem. This isolated filesystem is provided by an image, and the image must contain everything needed to run an application - all dependencies, configurations, scripts, binaries, etc. The image also contains other configurations for the container, such as environment variables, a default command to run, and other metadata.
- Get the app
- Before you can run the application, you need to get the application source code onto your machine.
- Clone the getting-started-app repositoryopen_in_new using the following command:
- $ git clone https://github.com/docker/getting-started-app.git
- View the contents of the cloned repository. You should see the following files and sub-directories.
- ├── getting-started-app/
- │ ├── package.json

- README.md
- | ├── spec/
- | ├── src/
- | └── yarn.lock
- [Build the app's image]
- To build the image, you'll need to use a Dockerfile. A Dockerfile is simply a text-based file with no file extension that contains a script of instructions. Docker uses this script to build a container image.
- In the getting-started-app directory, the same location as the package.json file, create a file named Dockerfile. You can use the following commands to create a Dockerfile based on your operating system.
- Mac / Linux Windows
- In the terminal, run the following commands.
- Make sure you're in the getting-started-app directory. Replace /path/to/getting-started-app with the path to your getting-started-app directory.
- $ cd /path/to/getting-started-app
- Create an empty file named Dockerfile.
- $ touch Dockerfile
- Using a text editor or code editor, add the following contents to the Dockerfile:
- *# syntax=docker/dockerfile:1*

- **FROM** node:18-alpine
- **WORKDIR** /app
- **COPY** . .
- **RUN** yarn install --production
- **CMD [**"node"**,** "src/index.js"**]**
- Build the image using the following commands:
- In the terminal, make sure you're in the getting-started-app directory. Replace /path/to/getting-started-app with the path to your getting-started-app directory.
- $ cd /path/to/getting-started-app
- Build the image.
- $ docker build -t getting-started
- Build the image using the following commands:
- In the terminal, make sure you're in the getting-started-app directory. Replace /path/to/getting-started-app with the path to your getting-started-app directory.
- $ cd /path/to/getting-started-app
- Build the image.
- $ docker build -t getting-started

- The docker build command uses the Dockerfile to build a new image. You might have noticed that Docker downloaded a lot of "layers". This is because you instructed the builder that you wanted to start from the node:18-alpine image. But, since you didn't have that on your machine, Docker needed to download the image.
- After Docker downloaded the image, the instructions from the Dockerfile copied in your application and used yarn to install your application's dependencies. The CMD directive specifies the default command to run when starting a container from this image.
- Finally, the -t flag tags your image. Think of this as a human-readable name for the final image. Since you named the image getting-started, you can refer to that image when you run a container.
- The . at the end of the docker build command tells Docker that it should look for the Dockerfile in the current directory.

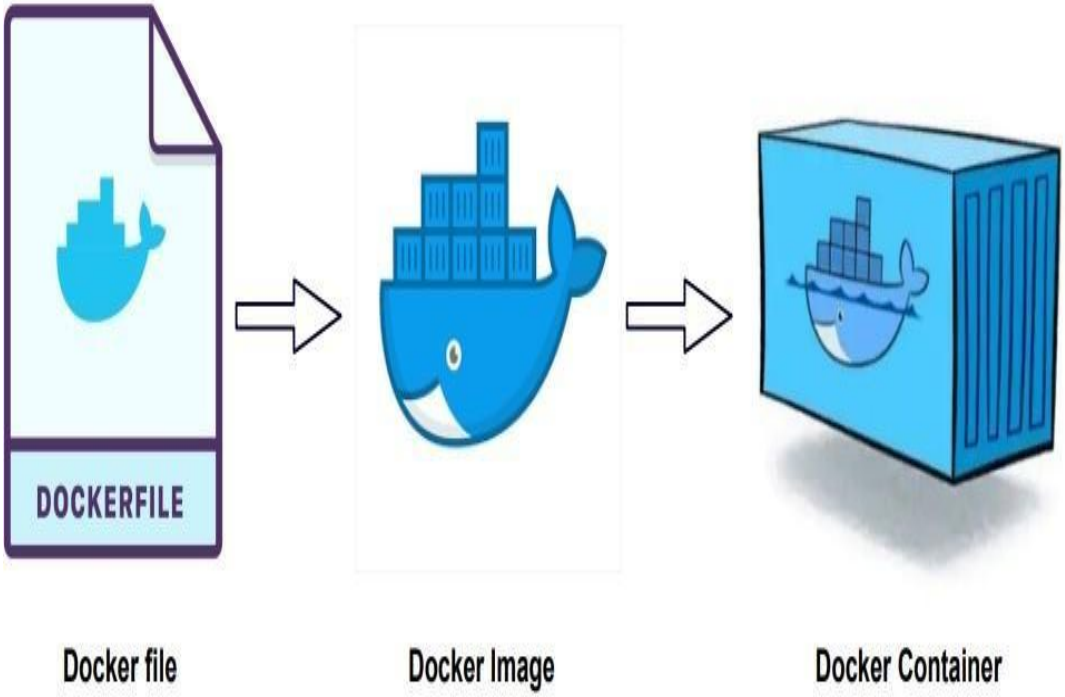
R.M.K GROUP OF INSTITUTIONS

# How Docker works?

- Docker Engine - Lightweight runtime toolset
- Doesn’t need a Hypervisor
- Similar to Process
- Packages your application into Image based on your specified Dockerfile
- Build your application into Images and pushes into Image Registry

R.M.K GROUP OF INSTITUTIONS

# How Docker works?



# Images

Fundamental building blocks of Container

Images serves as a template to

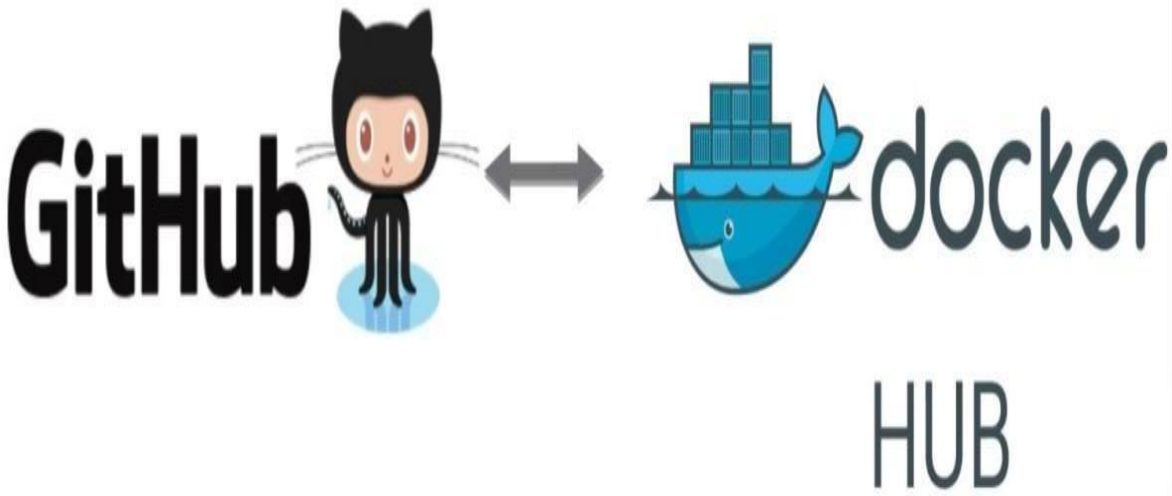
run your containerzed

applications Images are built

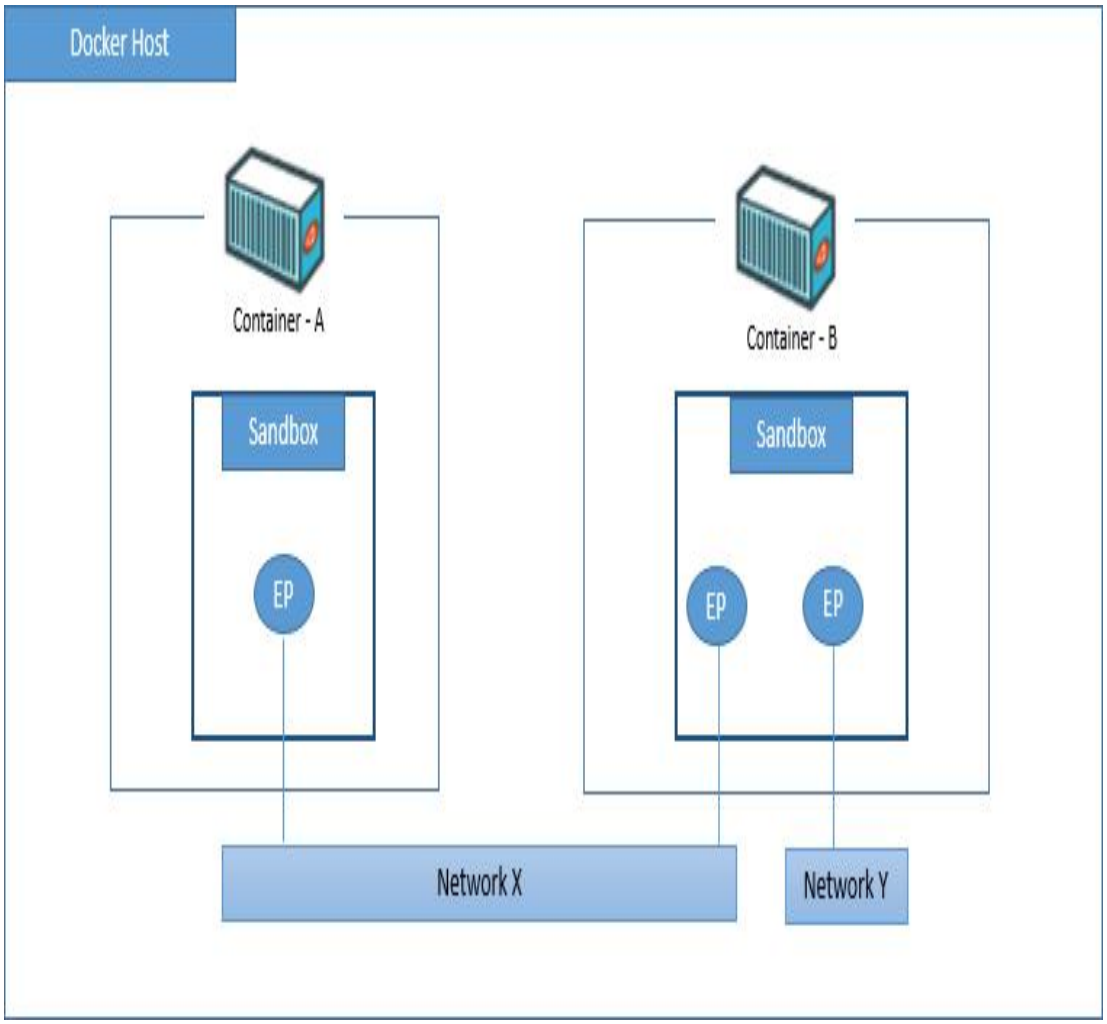
on layers.

layers are set
of file system
changes or
might be
commands/ins
tructions
specified in the
DockerFile

Push/Pull images instead of Source code!



# Docker links




R.M.K
GROUP OF
INSTITUTIONS

# Links

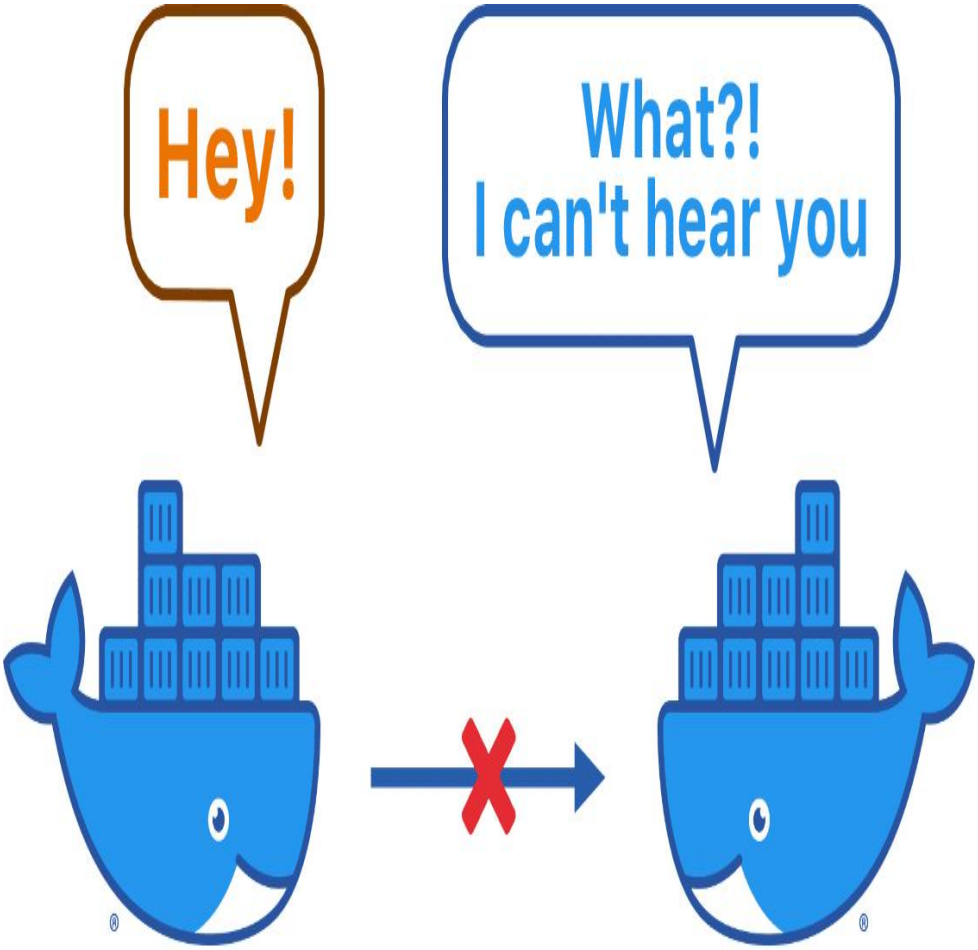
Docker links, also known as "container links" or "legacy container linking,"

Links are useful in enabling communication and dependency management between containers

Specify the source name of the container and it will set the environment variables in the destination Container to establish a connection

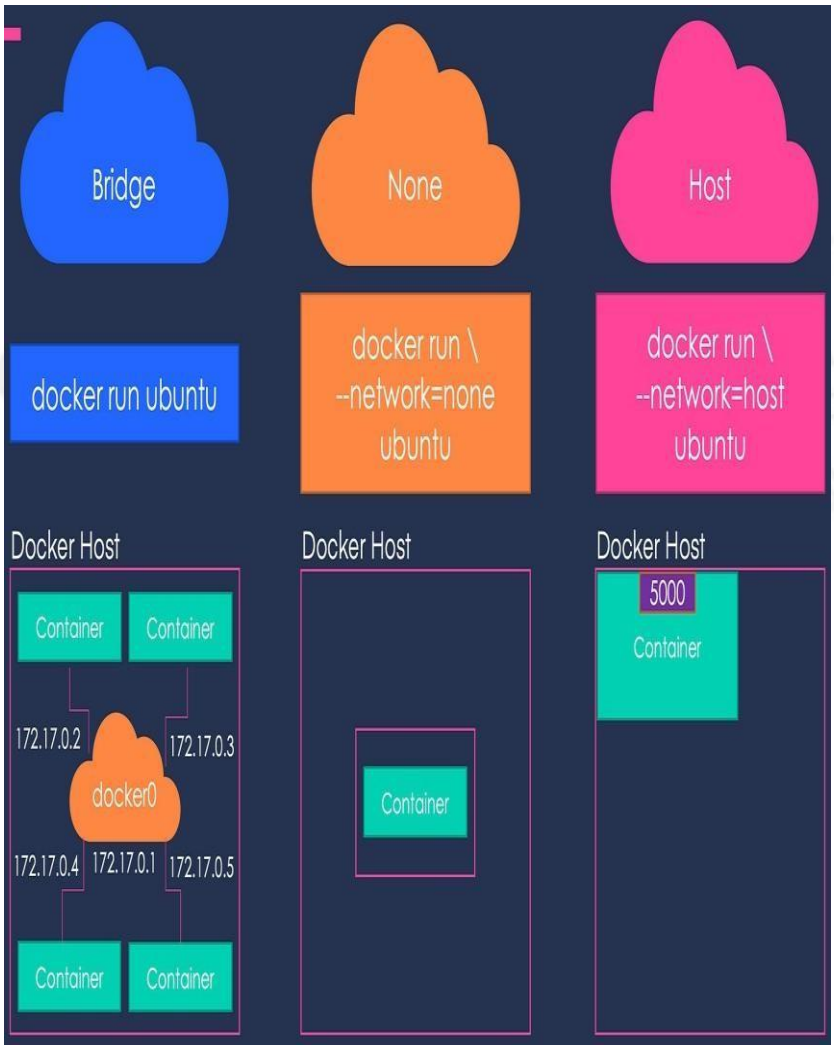
R.M.K GROUP OF INSTITUTIONS

# Networking in Docker

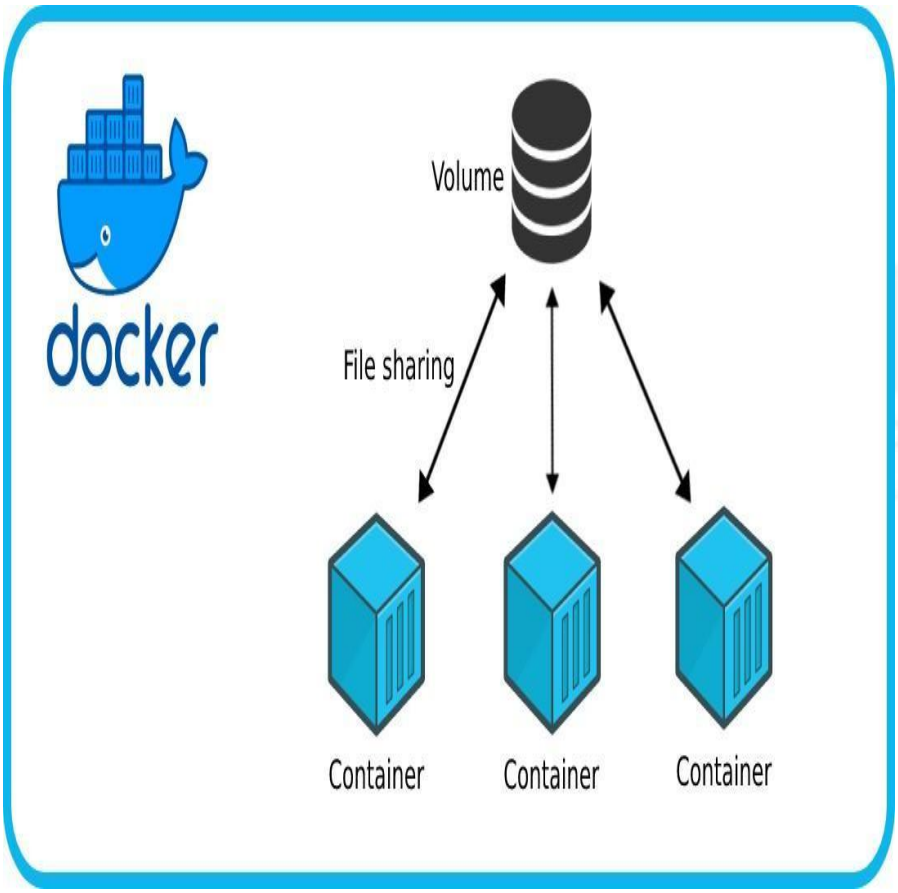



R.M.K GROUP OF INSTITUTIONS

# Docker Networking



# Docker Volume




docker volume create my-vol
docker volume ls

# Volumes:

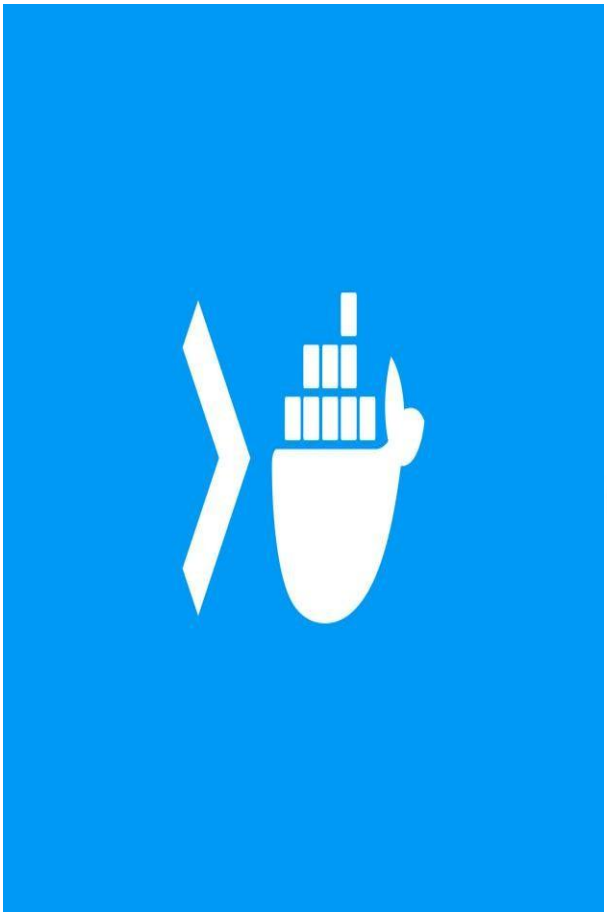
- Containers are non-persistent and the data is volatile
- Volumes are Virtual disks which can be attached a container to store data
- Types: Named volumes & Host mounted volumes

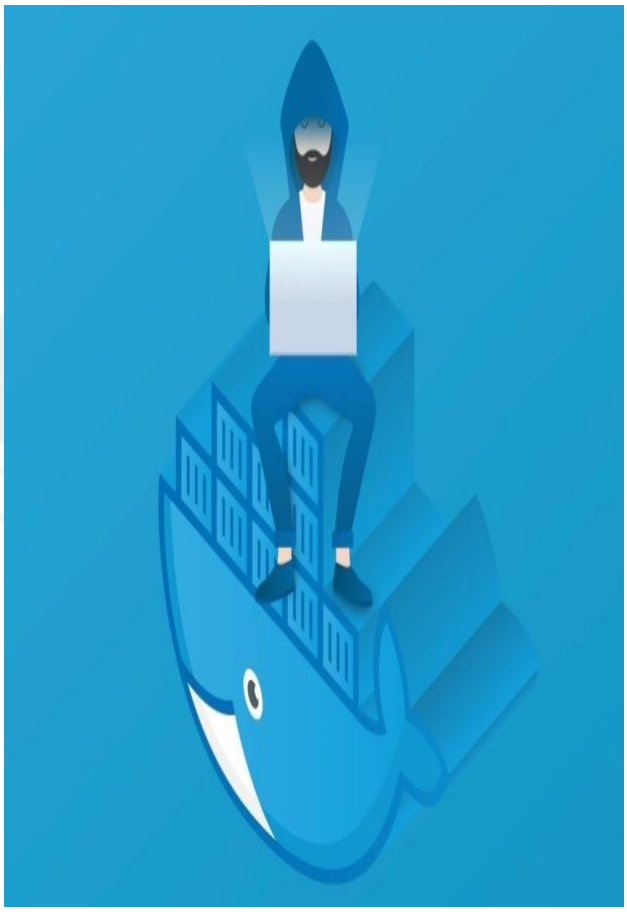
# Interactive Session – Exec:

- Create a interactive session in a container to interact with container shell or execute commands
- Docker exec command is used to start an interactive session

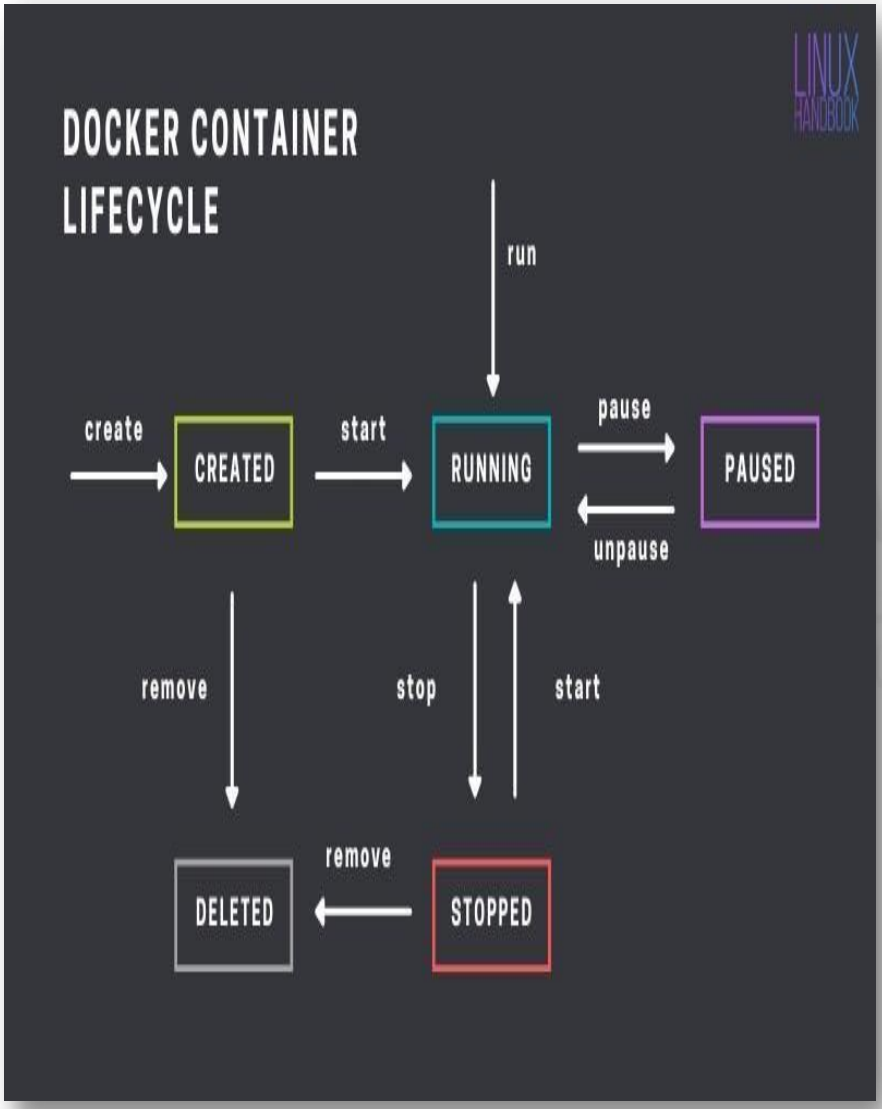
R.M.K GROUP OF INSTITUTIONS

# Docker Interactive terminal





docker exec -it
mycontainer sh




List containers

```
docker ps -all
```

Start container

```
docker run -it -d <image name>
```

Restart container

```
docker restart f8c52bedeecc
```

Stop container

```
docker stop f8c52bedeecc
```

List images

```
docker images
```

Pull image

```
docker pull image-name
```

Push image

```
docker push username/image-name
```

# Managing containers takes 108+ coffees

R.M.K GROUP OF INSTITUTIONS

# Docker REST API
# Introduction

The Docker REST API is an Application Programming Interface that allows developers and external applications to communicate with the Docker daemon programmatically. Instead of using Docker CLI commands such as docker run, docker build, or docker ps, applications can send HTTP requests to the Docker daemon through the REST API.

The API follows the REST (Representational State Transfer) architectural style, which uses standard HTTP methods such as GET, POST, DELETE, and PUT for interacting with Docker resources like containers, images, networks, and volumes.

Docker REST API is mainly used for:

- Automation of container operations
- Integration with DevOps tools
- Monitoring Docker environments
- Managing containers remotely
- The API is accessed through a Unix socket or TCP port where the Docker daemon listens for requests.

## Docker REST API Architecture

Docker uses a client–server architecture in which the Docker daemon exposes REST endpoints. External applications or the Docker CLI interact with the daemon through these APIs.

Components

**Docker Client**

- The command line tool (docker command).
- Sends API requests to Docker daemon.

**Docker REST API**

- A set of HTTP endpoints used for container management.

**Docker Daemon**

- Executes requests received through the API.

**Docker Objects**

- Containers
- Images
- Networks
- Volumes

R.M.K GROUP OF INSTITUTIONS

## Neat Diagram: Docker REST API Architecture

```
        +----------------------+
        |   User / Developer   |
        +----------+-----------+
                   |
                   | HTTP Request
                   v
        +----------------------+
        |   Docker Client      |
        |  (CLI / Applications)|
        +----------+-----------+
                   |
                   | REST API Calls
                   v
        +----------------------+
        |    Docker REST API   |
        |  (HTTP Endpoints)    |
        +----------+-----------+
                   |
                   v
        +----------------------+
        |     Docker Daemon    |
        |  (Container Engine)  |
        +----------+-----------+
                   |
    -------------------------------------
    |            |          |         |
    v            v          v         v
  Containers Images Networks  Volumes
```

## Working of Docker REST API

The working process of Docker REST API involves several steps.

### Step 1 – Client Request

The user or application sends an HTTP request to the Docker daemon.

Example:

GET /containers/json

### Step 2 – API Processing

The Docker REST API receives the request and validates it.

### Step 3 – Docker Daemon Execution

The Docker daemon performs the requested operation such as:

- creating containers
- listing images
- removing containers

### Step 4 – Response

The API sends a JSON response back to the client.

Example response:

```
[
 {
  "Id": "12345",
  "Image": "nginx",
  "Status": "running"
 }
]
```

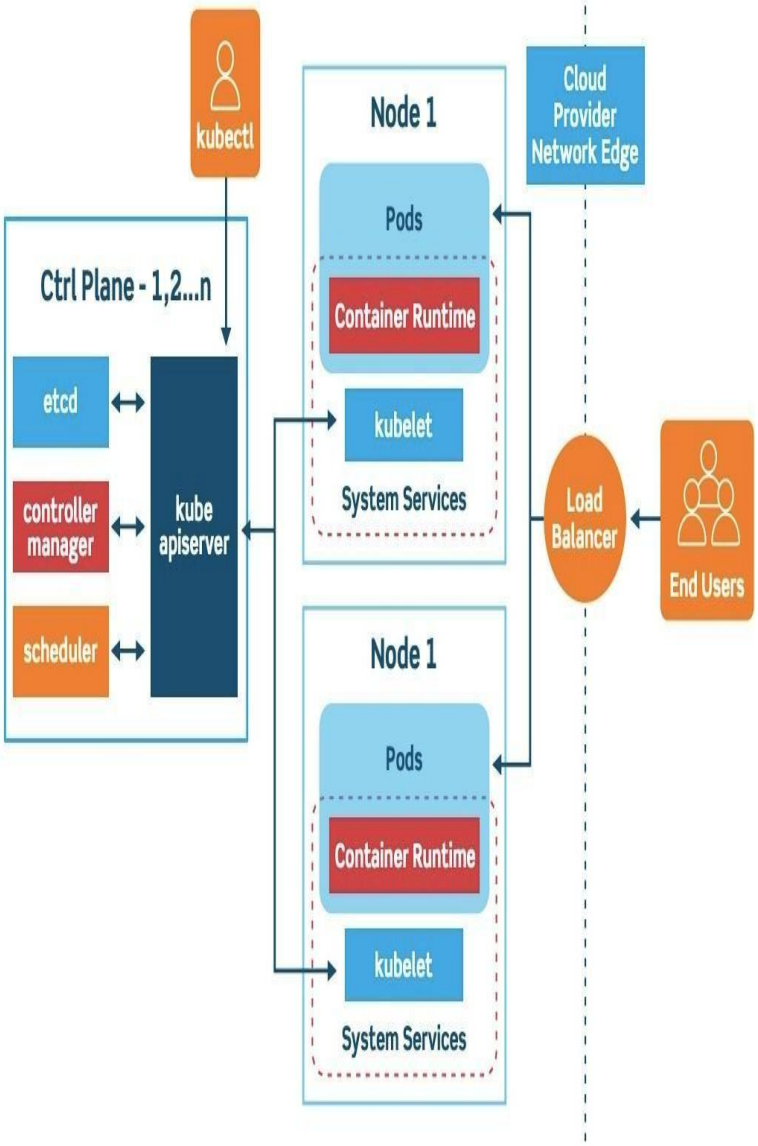
### Types of Docker REST API Operations

Docker REST API supports several operations using HTTP methods.

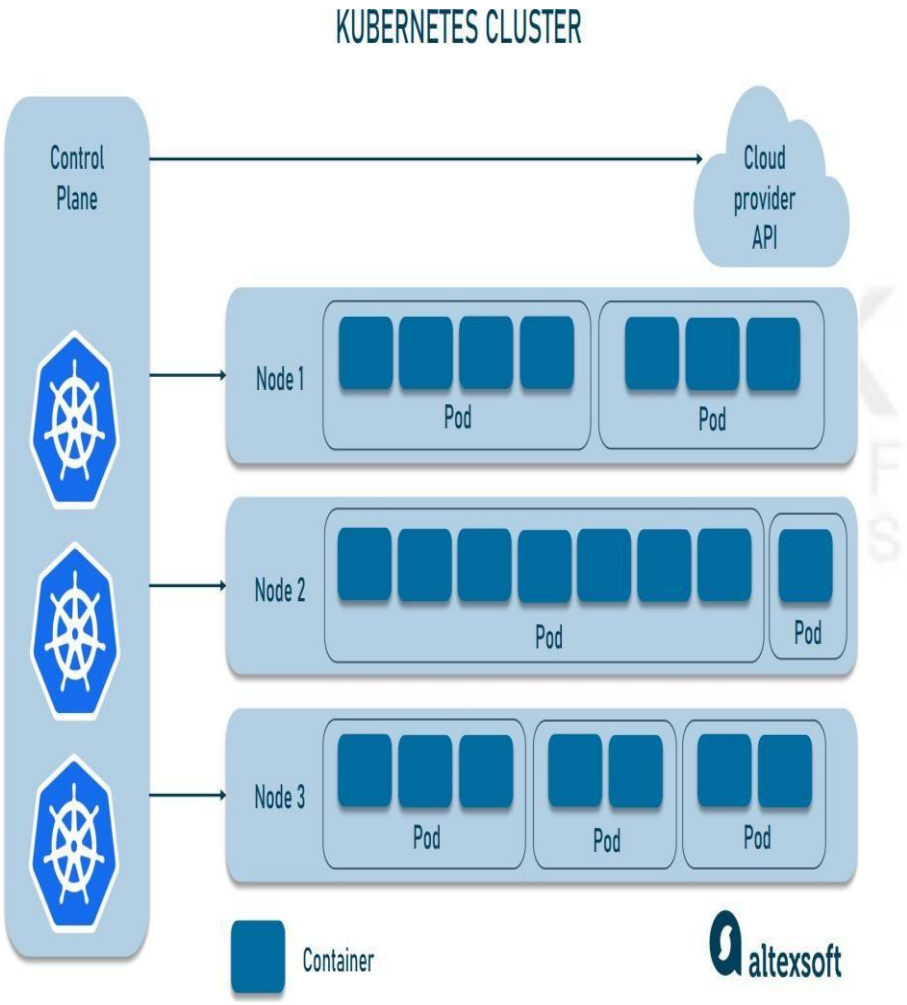
| HTTP Method | Operation | Example |
|---|---|---|
| GET | Retrieve information | List containers |
| POST | Create resources | Create container |
| DELETE | Remove resources | Delete container |
| PUT | Update resources | Update container settings |

# Kubernetes in a nutshell

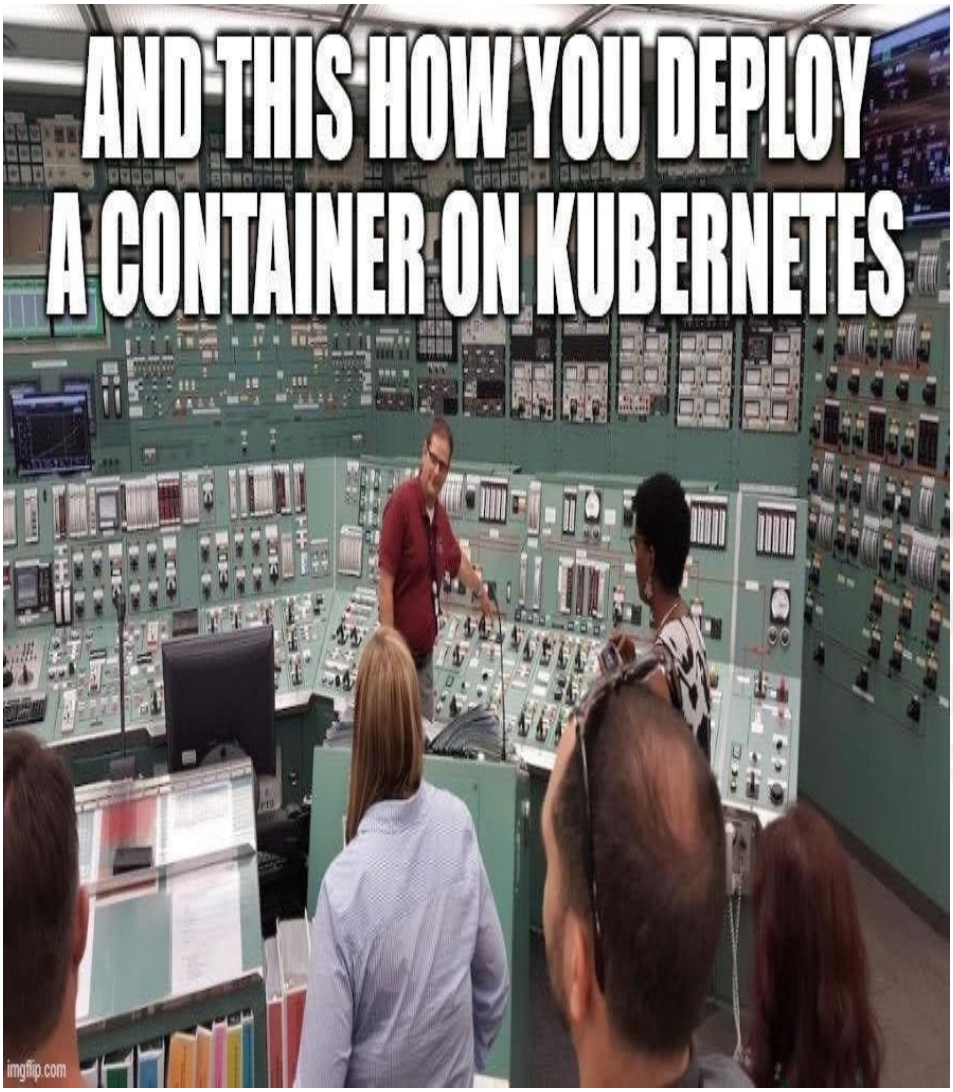
# Compo nents


kubectl
Node 1
Cloud Provider Network Edge
Pods
Ctrl Plane - 1,2...n
Container Runtime
etcd
kubelet
controller manager
kube apiserver
System Services
Load Balancer
End Users
scheduler
Node 1
Pods
Container Runtime
kubelet
System Services

# Architecture

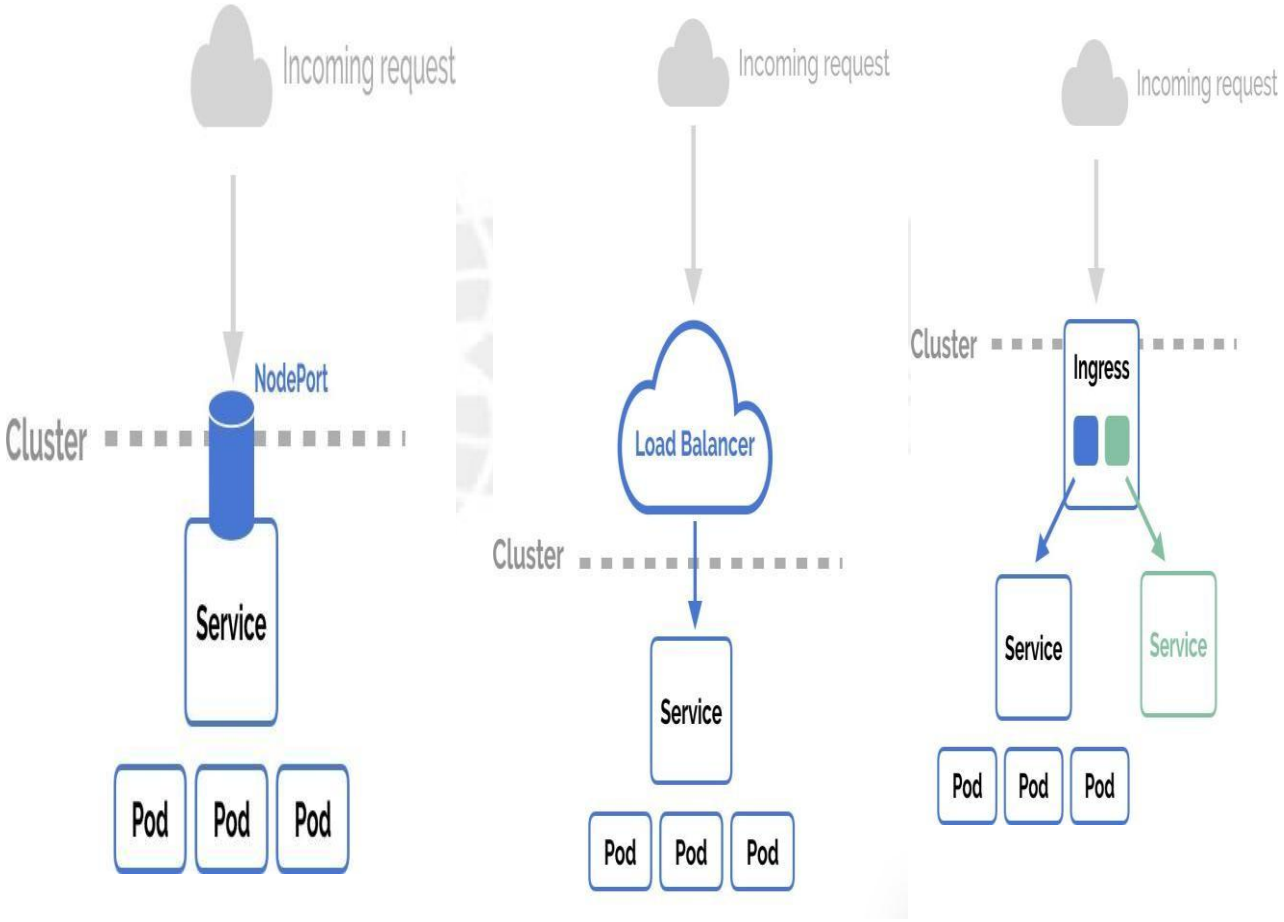


# As complex as it sounds!

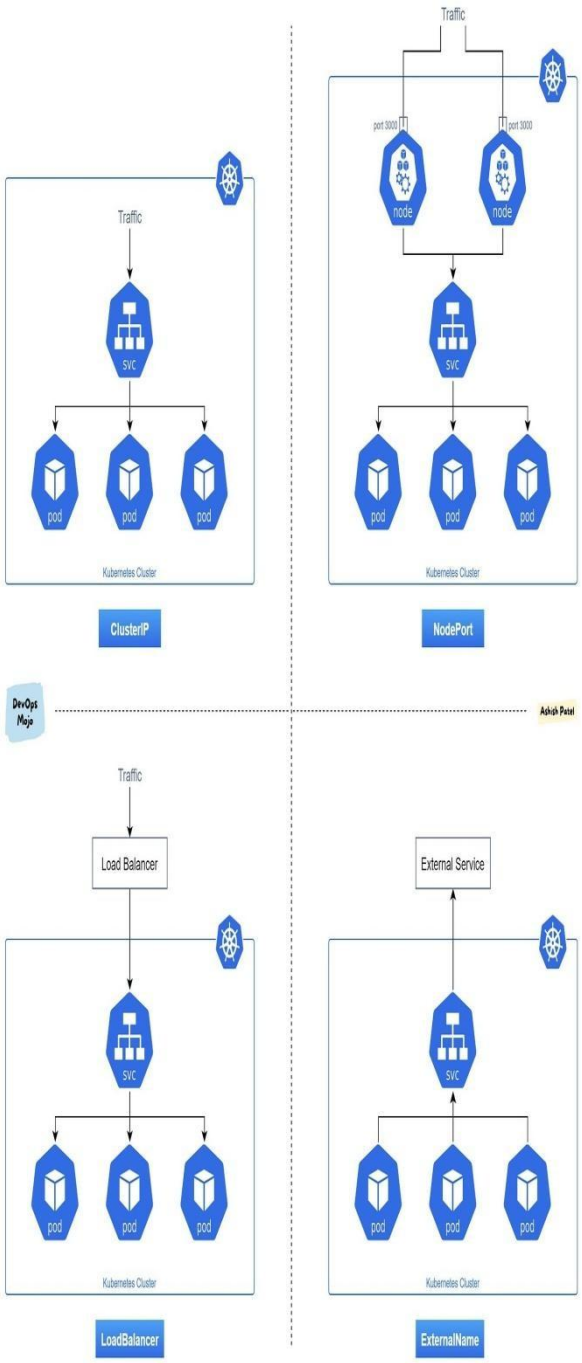



R.M.K GROUP OF INSTITUTIONS

# Services

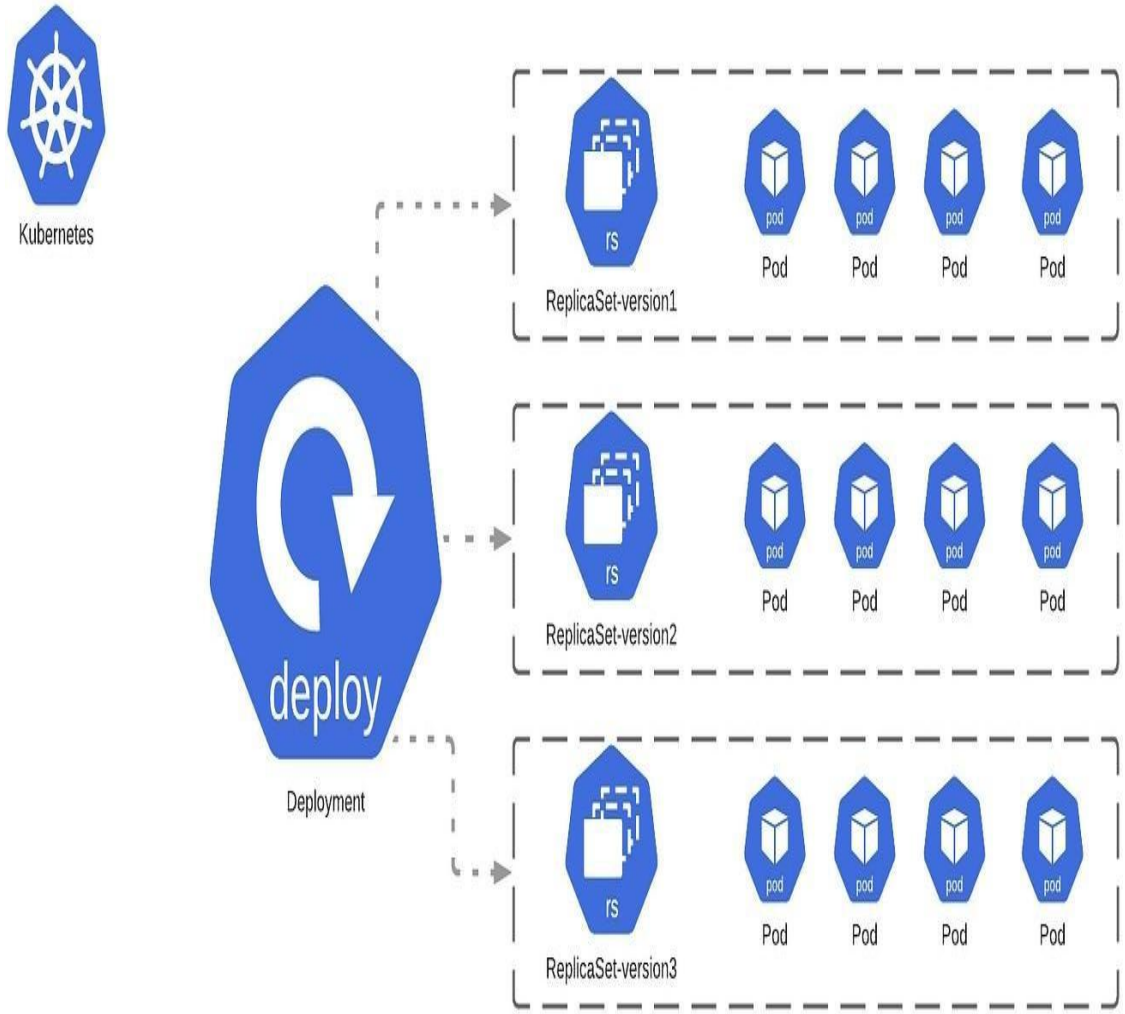

Incoming request
NodePort
Cluster
Service
Pod
Pod
Pod
Incoming request
Load Balancer
Cluster
Service
Pod
Pod
Pod
Incoming request
Cluster
Ingress
Service
Service
Pod
Pod
Pod

# Servi
# ces



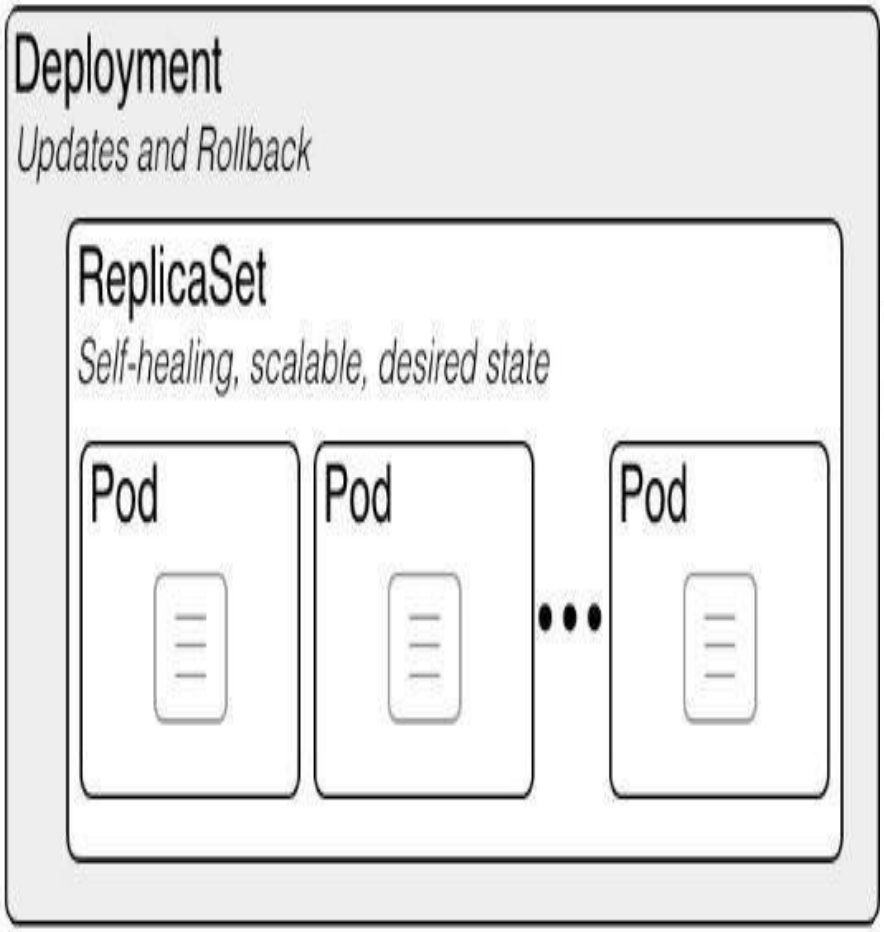

R.M.K GROUP OF INSTITUTIONS

# Replicaset and Deployments




R.M.K GROUP OF INSTITUTIONS

# Deployments vs Replicaset

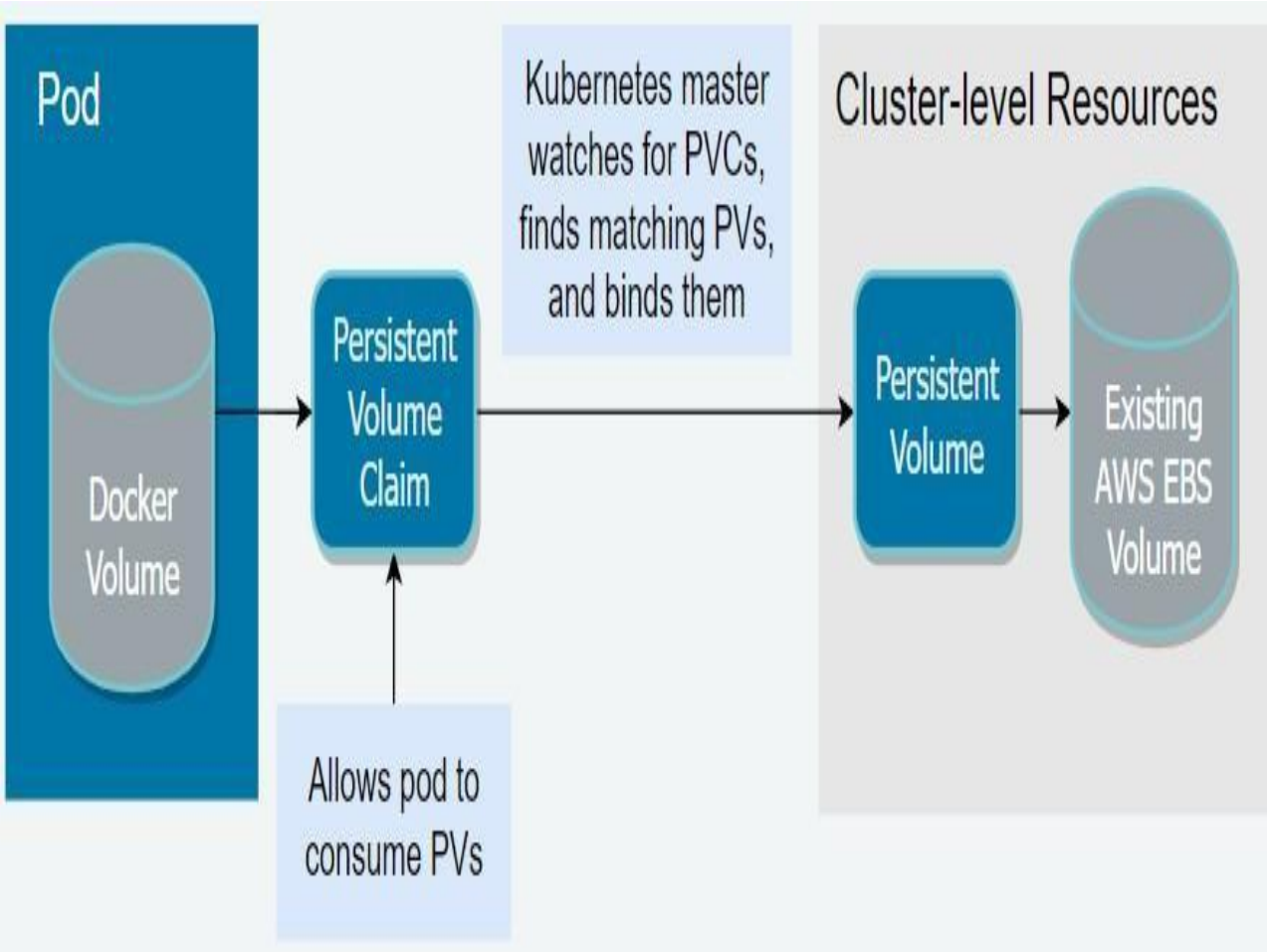



R.M.K
GROUP OF
INSTITUTIONS

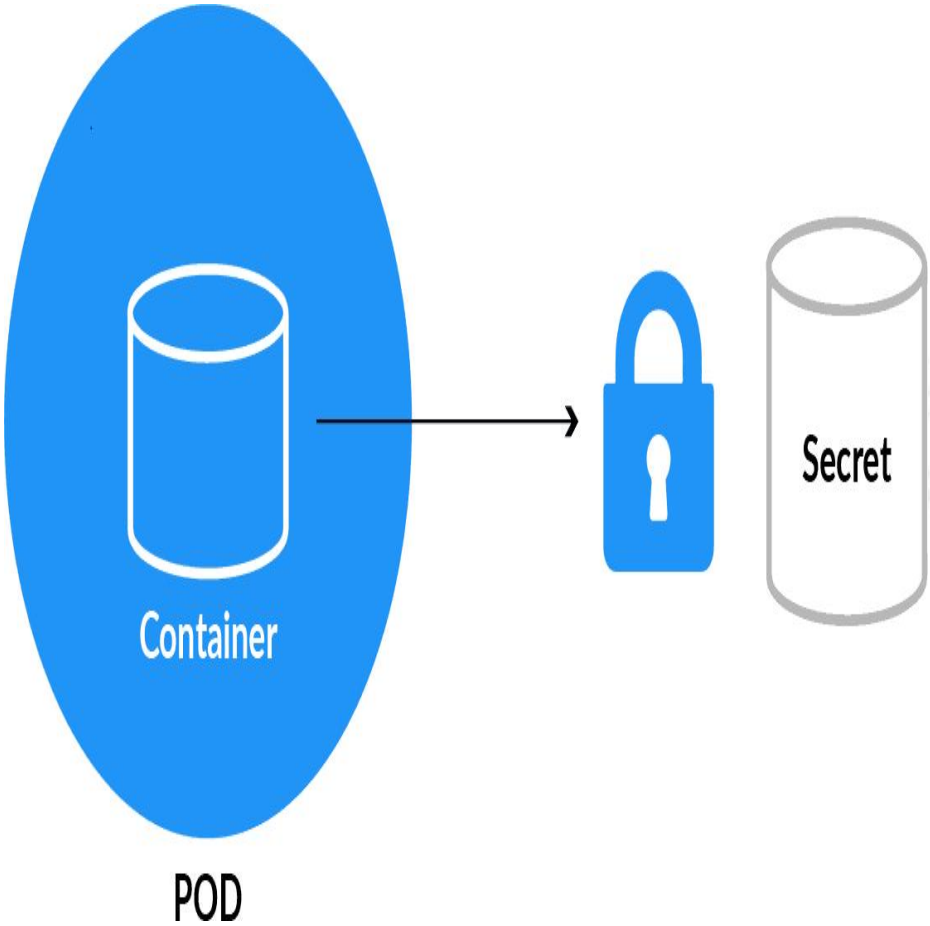
# Power of Deployments

R.M.K GROUP OF INSTITUTIONS

# Persistent Volume and claims



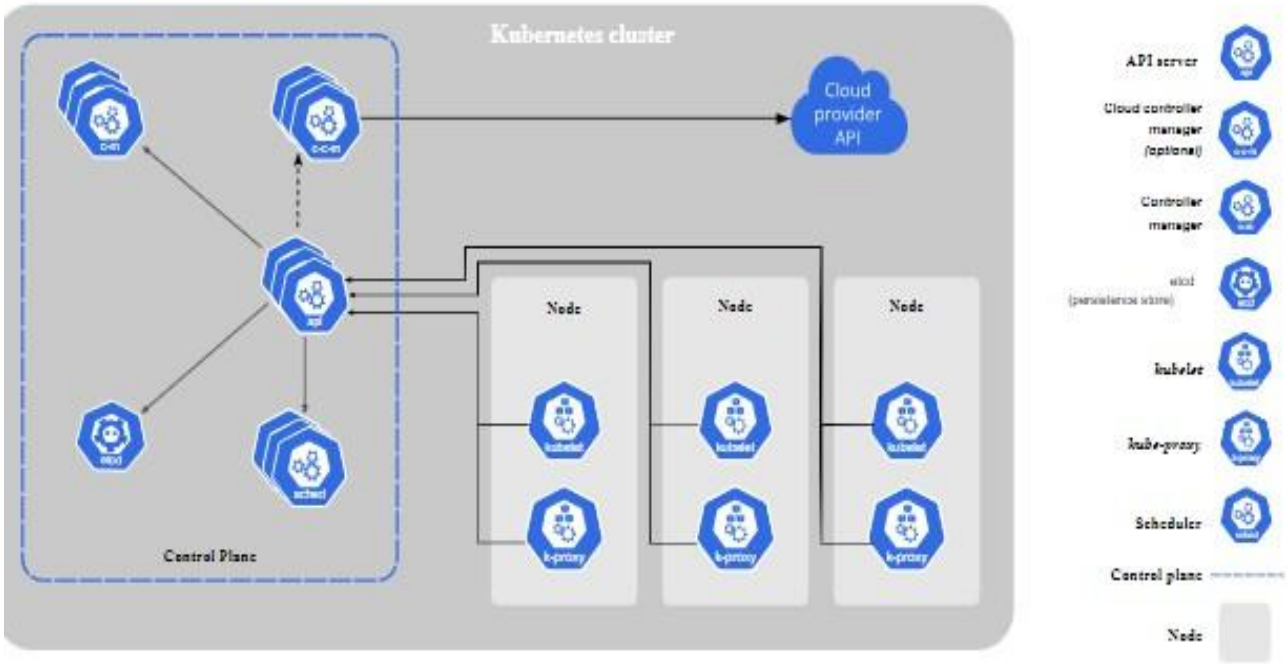
# Secrets




R.M.K
GROUP OF
INSTITUTIONS

# Kubernetes Components

- A Kubernetes cluster consists of a set of worker machines, called nodes, that run containerized applications. Every cluster has at least one worker node.
- The worker node(s) host the Pods that are the components of the application workload. The control plane manages the worker nodes and the Pods in the cluster. In production environments, the control plane usually runs across multiple computers and a cluster usually runs multiple nodes, providing fault-tolerance and high availability.




ne components of a Kubernetes cluster

- This document outlines the various components you need to have for a complete and working Kubernetes cluster.
- The components of a Kubernetes cluster
- Control Plane Components
- The control plane's components make global decisions about the cluster (for example, scheduling), as well as detecting and responding to cluster events (for example, starting up a new pod when a deployment's replicas field is unsatisfied).

- Control plane components can be run on any machine in the cluster. However, for simplicity, set up scripts typically start all control plane components on the same machine, and do not run user containers on this machine. See Creating Highly Available clusters with kubeadm for an example control plane setup that runs across multiple machines.
- kube-apiserver
- The API server is a component of the Kubernetes control plane that exposes the Kubernetes API. The API server is the front end for the Kubernetes control plane.
- The main implementation of a Kubernetes API server is kube-apiserver. kube-apiserver is designed to scale horizontally—that is, it scales by deploying more instances. You can run several instances of kube-apiserver and balance traffic between those instances.

- Kubernetes architecture and components
- A K8S cluster is made of a master node, which exposes the API, schedules deployments, and generally manages the cluster. Multiple worker nodes can be responsible for container runtime, like Docker or rkt, along with an agent that communicates with the master.
- Master components
- These master components comprise a master node:
- **Kube-apiserver.** Exposes the API.
- **Etcd.** Key value stores all cluster data. (Can be run on the same server as a master node or on a dedicated cluster.)
- **Kube-scheduler.** Schedules new pods on worker nodes.
- **Kube-controller-manager.** Runs the controllers.
- **Cloud-controller-manager.** Talks to cloud providers.
- Node components
- **Kubelet.** Agent that ensures containers in a pod are running.
- **Kube-proxy.** Keeps network rules and perform forwarding.
- **Container runtime.** Runs containers.

**Logs**
Docker logs are records of the output generated by applications running inside Docker containers. These logs help DevOps teams monitor container activity, detect errors, troubleshoot issues, and analyze application performance.

In Docker, logs mainly capture two types of output streams:

Standard Output (stdout) – normal messages generated by the application

Standard Error (stderr) – error messages or warnings

Docker automatically collects these outputs from containers and stores them using a logging driver.
Working of Docker Logs

When an application runs inside a container, it produces output messages. Docker captures these messages and stores them in log files or sends them to external logging systems.

Simple Architecture

```
Application inside Container
        |
    stdout / stderr
        |
        v
     Docker Engine
        |
    Logging Driver
        |
        v
  Log Storage / Monitoring Tools
```

Viewing Logs in Docker

Docker provides the docker logs command to view logs generated by a container.

**Syntax**

docker logs container_name

**Examples**

View container logs

```
docker logs mycontainer
```

View logs in real time

```
docker logs -f mycontainer
```

Show last 10 lines

```
docker logs --tail 10 mycontainer
```

**Logging Drivers in Docker**

Docker uses logging drivers to determine where logs are stored.
Some commonly used drivers include:

- json-file (default)
- syslog
- journald
- fluentd
- gelf
- awslogs

These drivers allow logs to be stored locally or sent to centralized logging systems.

## Importance of Docker Logs in DevOps

- Docker logs are important because they help in:
- Monitoring container performance
- Debugging application errors
- Tracking system activities
- Supporting continuous integration and deployment
- Centralized log analysis in DevOps environments

## Helm Framework

Introduction

The Helm Framework is a package manager for Kubernetes used to simplify the deployment and management of containerized applications. Although Helm is primarily used with Kubernetes, it is closely related to Docker-based applications, because Docker containers are the workloads deployed in Kubernetes clusters.

Helm helps developers and DevOps engineers install, configure, upgrade, and manage complex applications running in Kubernetes using predefined packages called Helm Charts.

Just as APT/YUM manage software packages in Linux, Helm manages Kubernetes applications.

Need for Helm in DevOps

In Kubernetes environments, deploying applications manually requires writing many configuration files such as:

- Pods
- Services
- Deployments
- ConfigMaps
- Secrets

Managing these YAML files manually becomes complex. Helm simplifies this process by providing predefined templates and reusable configurations.

Benefits of Helm

- Simplifies Kubernetes application deployment
- Reusable configuration templates
- Easy version management
- Simplifies upgrades and rollbacks
- Automates deployment process

Helm Architecture

- Helm follows a simple architecture consisting of:
- Helm Client
- Helm Charts
- Helm Repository
- Helm Releases
- Kubernetes Cluster

```
+-----------------+
|    Developer    |
+---------+--------+
          |
          v
+-----------------+
|    Helm CLI     |
| (Helm Client)   |
+---------+--------+
          |
          v
+-----------------+
|  Helm Charts    |
| (Application    |
| Templates)      |
+---------+--------+
          |
          v
+-----------------+
| Kubernetes API  |
+---------+--------+
          |
          v
+---------------+
| Kubernetes    |
| Cluster       |
| Pods, Services |
| Deployments   |
+---------------+
```

Helm Workflow

The workflow of Helm involves the following steps:

1.Developer creates or downloads a Helm chart.

2.Helm installs the chart into Kubernetes.

3.Kubernetes creates resources like Pods and Services.

4.Helm tracks the deployment as a Release.

Helm Repository

A Helm Repository is a collection of Helm charts stored in a remote location.

It allows users to share and download application packages.

Popular Helm Repositories

•Bitnami Helm Charts

•Artifact Hub

•Kubernetes official charts

Example repository command:

helm repo add bitnami https://charts.bitnami.com/bitnami

Helm Releases

A Release is an instance of a Helm chart running in a Kubernetes cluster.

Each installation of a chart creates a new release.

Example:

helm install myapp nginx-chart

Here:

myapp → Release name

nginx-chart → Helm chart

Helm keeps track of all releases so that they can be upgraded or rolled back.

Helm Commands

Some important Helm commands used in DevOps are:

**Install a Chart**

helm install myrelease chartname

**List Installed Releases**

helm list

**Upgrade Application**

helm upgrade myrelease chartname

**Rollback Release**

helm rollback myrelease 1

**Uninstall Release**

helm uninstall myrelease

R.M.K GROUP OF INSTITUTIONS

## Single Mode and Cluster Mode

### Single Mode (Single Node Cluster)

Single Mode refers to a Kubernetes setup where both the master node and worker node run on the same machine. It is also called a single-node Kubernetes cluster.

This configuration is mainly used for:

- Learning Kubernetes
- Development and testing
- Local experiments

Tools like Minikube or Docker Desktop Kubernetes commonly run Kubernetes in single mode.

Architecture of Single Mode

In this mode, all Kubernetes components run on one system.

Components in Single Mode

Master Node Components

- API Server
- Scheduler
- Controller Manager

Worker Node Components

- Kubelet
- Container Runtime (Docker / containerd)
- Pods

Since it is a single node, the same machine performs both roles.

### Advantages of Single Mode

- Easy to install
- Low hardware requirement
- Suitable for development
- Ideal for testing Kubernetes features

### Limitations of Single Mode

- No high availability
- Not suitable for production
- Limited scalability
- Single point of failure

### Cluster Mode (Multi-Node Kubernetes Cluster)

Cluster Mode refers to a Kubernetes setup where multiple machines (nodes) work together to run containerized applications.

It contains:

One or more master nodes

Multiple worker nodes

Cluster mode is used in production environments where scalability and reliability are required.

R.M.K GROUP OF INSTITUTIONS

Architecture of Cluster Mode

Cluster mode distributes tasks across multiple nodes.

**Master Node Responsibilities**

- Cluster management
- Scheduling containers
- Managing API requests
- Maintaining cluster state

**Worker Node Responsibilities**

- Running application containers
- Managing pods
- Communicating with master node

```
              +--------------------+
              |    Master Node     |
              |--------------------|
              | API Server         |
              | Scheduler          |
              | Controller Manager |
              +---------+----------+
                        |
        ----------------------------------------
        |               |                      |
        v               v                      v
  +-------------+   +-------------+   +-------------+
  | Worker Node |   | Worker Node |   | Worker Node |
  |-------------|   |-------------|   |-------------|
  | Kubelet     |   | Kubelet     |   | Kubelet     |
  | Container   |   | Container   |   | Container   |
  | Runtime     |   | Runtime     |   | Runtime     |
  | Pods        |   | Pods        |   | Pods        |
  +-------------+   +-------------+   +-------------+
```

**Features of Cluster Mode**

- High availability
- Load balancing
- Fault tolerance
- Scalability
- Distributed processing

**Advantages of Cluster Mode**

- High Reliability
- If one node fails, others continue running.
- Scalability
- More nodes can be added to increase capacity.
- Load Distribution
- Workloads are distributed across nodes.
- Better Resource Utilization
- CPU and memory are efficiently used.

# Part A

| |
|---|
| What is Docker container and how to create it? |
| Difference between Docker Swarm and k8s? |
| How are Kubernetes and Docker related? |
| Explain Nodes and cluster in Kubernetes? |
| What are Kubernetes and how Docker related? |
| What is container orchestration? |
| What is K8s |
| What is the difference between deploying applications on hosts and containers? |
| What are the features of Kubernetes? |
| What is a node in Kubernetes? |
| What process runs on Kubernetes Master Node? |
| What is a pod in Kubernetes? |
| What is a cluster of containers in Kubernetes? |
| What are Daemon sets? |

# Part B

- Explain in Docker networking in detail
- Explain the Docker architecture with neat diagrams
- Explain the purposes of up, run, and start commands of Docker compose.
- Explain the difference between Docker image and layer
- What is the default Docker network driver? How can you change it when running a Docker image?
- Explain the difference between Deployments and Replica set in detail
- Discuss in detail about Persistent Volume and claims
- Explain the difference between Deployments and Replica set in detail
- How does Kubernates help in the continuous monitoring of systems, applications, and services?

R.M.K GROUP OF INSTITUTIONS

# SUPPORTIVE ONLINE COURSES – UNIT 4

- https://infyspringboard.onwingspan.com/web/en/app/toc/lex_auth_013844070369165312492 79_shared/overview

R.M.K GROUP OF INSTITUTIONS

# REAL TIME APPLICATION- UNIT 4

- Explain the Case study on Docker Implementation for a Smart Retail Digital Signage Solution .
- Design a Docker Implementation for a Smart Amazon website Solution
- Develop Software Development in Docker for Surgical Instrument Tracking

R.M.K GROUP OF INSTITUTIONS

## SOFTWARE PROJECT MANAGEMENT

Software project management refers to the branch of project management dedicated to the planning, scheduling, resource allocation, execution, tracking and delivery of software and web projects.

Project management in software engineering is distinct from traditional project management in that software projects have a unique lifecycle process that requires multiple rounds of testing, updating, and customer feedback. Most IT-related projects are managed in the agile style, in order to keep up with the increasing pace of business, and iterate based on customer and stakeholder feedback.

### ❁ ProWorkflow

ProWorkflow is a web-based project management software that enables users to manage tasks and projects, track time, organize contacts, and generate reports for their business. It is a productivity application that provides a comprehensive set of features, yet still easy to use for all members of the team. Aside from the great tools and functionality from the software, customers also enjoy free quality support through consultations and trainings that help get their businesses up and running.

### ❁ Jira

Jira is an agile project management software used by development teams to plan, track, and release software. It is a popular tool designed specifically and used by agile teams. Aside from creating stories, planning sprints, tracking issues, and shipping up-to-date software, users also generate reports that help improve teams, and create their own workflows. As part of Atlassian, it integrates with many tools that enable teams to manage their projects and products from end to end. Jira Software is built for every member of your software team to plan,track, and release great software. Every team has a unique process for shipping software. Use an out-of-the-box workflow, or create one to match the way your team works.

## REFERENCES:


- Jennifer Davis and Ryn Daniels, Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling at Scale, First Edition, O'Reilly Publications.
- Gene Kim, Patrick Debois et al., The DevOPS Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations, IT Revolution Press; Illustrated edition, 2016.

# ASSESSMENT SCHEDULE

## Tentative schedule for the Assessment During 2025-2026 Even semester

| S.NO | Name of the Assessment | Start Date | End Date | Portion |
|---|---|---|---|---|
| 1 | UNIT TEST 1 | | | UNIT 1 |
| 2 | IAT 1 | | | UNIT 1 & 2 |
| 3 | UNIT TEST 2 | | | UNIT 3 |
| 4 | IAT 2 | | | UNIT 3 & 4 |
| 5 | MODEL | | | ALL 5 UNITS |

R.M.K GROUP OF INSTITUTIONS

# MINI PROJECT SUGGESTIONS

- **Exercise 1:**
- Install Groovy
- Create a Pipelines in Jenkins which will perform following steps
  - Configure Java/Maven/Private Repo
  - Git Clone of the above said project
  - Maven Build
  - Maven Test
  - Sonar Scanner (if quality if OK then proceed, else exit with error message)
  - Repo Setup and Install Libraries in the Repo
  - Docker Clean Containers
  - Docker Clean Images
  - Docker Build
  - Docker Run Container
- Points to be observed: -
- Build Stability
- Finger Prints
- Test Case Results
- Workspace
- Application Logs
- Jenkins Pre / Post Actions
- Email Notifications
- Sonar Qube Report Analysis
- Time Taken for each Steps
- Process Refinement

R.M.K GROUP OF INSTITUTIONS

# Thank you

**Disclaimer:**

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.